

CSc 453 — Assignment 4

Christian Collberg
University of Arizona, Department of Computer Science

February 26, 2026

Extend the parser from assignment 3 to build an abstract syntax tree (AST). Your compiler should be invoked like this:

```
$ luca_ast prog1.luc prog1.xml prog1.gv
```

The result is a file `prog1.xml` which is the AST in XML format. Here's an example.

<pre>PROGRAM P; BEGIN WRITE a+b; END.</pre>	$\Rightarrow$	<pre><block> <PROGRAM name="P" pos="1"> <DECLNULL pos="2"/> <STATS pos="3"> <WRITE pos="3"> <BINARY op="PLUS" pos="3"> <VARREF ident="a" pos="3"> <DESNUL pos="3"/> </VARREF> <VARREF ident="b" pos="3"> <DESNUL pos="3"/> </VARREF> </BINARY> </WRITE> <STATNULL pos="4"/> </STATS> </PROGRAM> </block></pre>
---	---------------	---

You should also generate an Graphviz file `prog1.gv` allowing you to visualize the generated tree.

Appendix A gives the definition of the abstract syntax you should produce.

1 Getting Started

This assignment can be done in teams of one or two.

To get started, follow these instructions:

1. Unzip `assignment-4.zip` which gives you these files:

```
assignment-4/assignment-4.pdf  
assignment-4/validator.sh  
assignment-4/firstname_lastname/lexer/  
assignment-4/firstname_lastname/parser/  
assignment-4/firstname_lastname/ast/*.java  
assignment-4/firstname_lastname/tests/*.luc  
assignment-4/firstname_lastname/tests/*.xml  
assignment-4/firstname_lastname/collaboration.json  
assignment-4/firstname_lastname/survey.json
```

2. Then, assuming your name is Pat Jones, execute these commands:

```
> cd assignment-4
> mv firstname_lastname pat_jones
```

If you are working in a group of two, pick one of your names.

3. Now, go ahead and solve the problems, entering your answers in the template files in `pat_jones/`.
4. AST node definitions have been provided for you in `assignment-4/firstname_lastname/ast/*.java`.
5. You should copy your code from the previous assignments into `assignment-4/firstname_lastname/lexer/` and `assignment-4/firstname_lastname/parser/`.
6. Code for generating Graphviz files has been provided for you. Call it like this:

```
ast.PROGRAM e = ...
ast.Graphviz.clear();
e.toGraphviz();
ast.GraphvizToFile("AST",graphvizFileName);
```

Don't show your code to anyone outside your team, don't read anyone else's code, don't discuss the details of your code with anyone. If you need help with the assignment see the TA or the instructor.

2 Implementation Notes

- This assignment shall be coded in Java.
- You must provide a shell script `luca_ast` that calls Java with the appropriate arguments.
- You can use the `xmllint` program to check the syntactic correctness of the generated XML:

```
$ xmllint test1_ast.xml
```

- A large number of Java classes for building abstract syntax trees have been provided for you. Even though the output of the program is an XML file, internally, the compiler must first build an explicit tree using these AST classes, and then walk that tree to print out the XML.
- See Sections 4.1.2 and 4.2.4 in *Kenneth C. Louden, Compiler Construction, Principles and Practice* for a description on how to adapt top-down parsers to build abstract syntax trees.

2.1 Grading

There are 58 test cases in `assignment-4/firstname_lastname/tests/`. For each case that does not produce an error or crashes, but produces a good AST, you will get 1.5 points. You can earn an additional 13 points from more complicated secret test cases.

You can see some of the test cases in Appendix B.

3 Submission

To submit, follow these instructions:

1. Verify that your submission contains a working **Makefile** and *all* the files necessary to build the **lexer and parser**. Your program must compile out of the box. The graders will *not* try to debug your program or your makefile for you.
2. Fill out `collaboration.json` with information about your group.
3. Optionally fill out `survey.json` to give me feedback about the assignment.
4. Package up your answers and verify that they look OK:

```
> zip -r pat_jones.zip pat_jones
> assignment-4/validator.sh pat_jones.zip
==== This is the LigerLabs assignment validator ====
...      ...      ...      ...
==== 6: Checking that all problems have been answered.
==== 7: Checking that all answer files have valid structure.
```

5. Upload `pat_jones.zip` to d2l. If you are working in a group of two, you should only submit once.

A Abstract Syntax

Below are the nodes in the Luca abstract syntax tree. Node-names are in **bold**, attributes are in a **typewriter** font, and node references (subtrees) are in *italics*.

A.1 Declarations

(PROGRAM *Ident* *Pos* **Decls** **Stats**)

This is the topmost node of any AST. **Ident** is the name of the program, **Pos** the line number where **Ident** occurs. *Decls* and *Stats* are the sub-trees for declarations and statements, respectively. Here's an example:

<pre>PROGRAM P; BEGIN END.</pre>	$\Rightarrow$	<pre><block> <PROGRAM name="P" pos="1"> <DECLNULL pos="2"/> <STATNULL pos="3"/> </PROGRAM> </block></pre>
----------------------------------	---------------	---

(DECLS *Pos* **Left** **Right**)

A list of declarations are linked together using **DECLS** nodes. *Left* points to the actual declaration, *Right* is the remaining subtree of declarations.

(DECLNULL *Pos*)

This node ends a sequence of declarations.

(PROCDECL *Ident* *Pos* **Formals** **Decls** **Stats**)

This is the topmost node of a procedure declaration. *Formals* is a list of **FORMALDECL** nodes. *Decls* and *Stats* are the sub-trees for declarations and statements, respectively.

(FORMALDECL *Ident* *TypeName* *Mode* *Pos*)

Ident is the name of a formal parameter, *TypeName* its type. *Mode* is **VAL** or **VAR**. **FORMALDECLS** are linked together using **DECL** nodes to form the lists of formal declarations used in **PROCDECLS**.

(**VARDECL** *Ident* *TypeName* *Pos*)

VARDECLs are linked together using **DECL** nodes to form lists of variable declarations. *Ident* is the name of the variable, *TypeName* its type.

(**CONSTDECL** *Ident* *TypeName* *Pos* *Expr*)

Ident is the name of the constant, *TypeName* its type, and *Expr* the root of an expression tree.

(**ARRAYDECL** *Ident* *ElementType* *Pos* *Count*)

ARRAYDECL nodes represent an array type declaration. *Ident* is the name of the type, *Count* is the root of an expression tree computing the number of elements in the array, and *ElementType* the type of the elements of the array.

(**RECORDDECL** *Ident* *Pos* *Fields*)

RECORDDECL nodes represent a record type declaration. *Ident* is the name of the type, and *Fields* is a list of **FIELDDECL** nodes, linked together on **DECL**s.

(**FIELDDECL** *Ident* *TypeName* *Pos*)

FIELDDECL nodes represent a field in a record. They are linked together in a list of **DECL** nodes, terminated by a **DECLNULL**.

A.2 Statements

(**STATS** *Pos* *Left* *Right*)

A list of statements are linked together using **STAST** nodes. *Left* points to the actual statement, *Right* is the remaining subtree of statements. Example:

<pre> PROGRAM P; BEGIN WRITELN; WRITELN; WRITELN; END. </pre>	$\Rightarrow$	<pre> <block> <PROGRAM name="P" pos="1"> <DECLNULL pos="2"/> <STATS pos="3"> <WRITELN pos="3"/> <STATS pos="4"> <WRITELN pos="4"/> <STATS pos="5"> <WRITELN pos="5"/> <STATNULL pos="6"/> </STATS> </STATS> </STATS> </PROGRAM> </block> </pre>
---	---------------	---

(**STATNULL** *Pos*)

This node ends a sequence of statements.

(**PROCCALL** *Pos* *Des* *Actuals*)

PROCCALL nodes represent a procedure call statement. *Des* is a designator representing the procedure to be called (always just one **VARREF** for this version of LUCA), and *Actuals* is a tree of **ACTUAL** nodes, the actual arguments to the call.

(**ACTUAL** *Pos* *Expr* *Next*)

A tree of **ACTUAL** nodes are used to represent the argument list to a procedure call. *Expr* is the root of an expression tree, *Next* refers to another **ACTUAL** node or **ACTUALNULL**.

(**ACTUALNULL** *Pos*)

This node ends a sequence of expressions.

(WRITE Pos *Expr*)

Expr is the root of an expression tree whose value is to be written.

(WRITELN Pos)

A node with no children and no input attributes other than Pos.

(READ Pos *Des*)

Read is the root of a designator list representing the variable into which the value is supposed to be read.

(IF1 Pos *Expr Then*)

Expr is the root of an expression tree. *Then* is the body of the “then” part of the statement.

(IF2 Pos *Expr Then Else*)

Expr is the root of an expression tree. *Then* and *Else* are the bodies of the “then” and “else” parts of the statement.

(WHILE Pos *Expr Stats*)

Expr is the root of the expression tree for the loop condition. *Stats* is the body of the loop.

(REPEAT Pos *Expr Stats*)

Expr is the root of the expression tree for the loop condition. *Stats* is the body of the loop.

(LOOP Pos *Stats*)

Stats is the body of the loop.

(EXIT Pos)

This node represents the EXIT statement which can only occur within the body of a LOOP statement.

(ASSIGN Pos *Des Expr*)

Expr is the expression tree for the right hand side of the statement, *Des* represents the designator for the left and side.

A.3 Expressions and Designators

(INTLIT Value Pos)

Value is a decimal integer literal.

(CHARLIT "Value" Pos)

Value is a one-character string.

(STRINGLIT "Value" Pos)

Value is a string.

(REALLIT Value Pos)

Value is a floating-point number.

(VARREF Ident Pos *Next*)

VARREFs are the root of a designator list. This list begins with a **VARREF**, and ends with a **DESNUL**. Inbetween are **INDEX** and **FIELDREF** nodes representing record field and array references. Example’:

```

PROGRAM P;
BEGIN
  READ a.b[c];
END.

```



```

<block>
<PROGRAM name="P" pos="1">
  <DECLNULL pos="2"/>
  <STATS pos="3">
    <READ pos="3">
      <VARREF ident="a" pos="3">
        <FIELDREF ident="b" pos="3">
          <INDEX pos="3">
            <VARREF ident="c" pos="3">
              <DESNNULL pos="3"/>
            </VARREF>
          <DESNNULL pos="3"/>
        </INDEX>
      </FIELDREF>
    </VARREF>
  </READ>
  <STATNULL pos="4"/>
</STATS>
</PROGRAM>
</block>

```

(DESNNULL Pos)

The last node of a designator list.

(INDEX *Index Next* Pos)

INDEX nodes represent an array index reference. *Index* is the root of an expression tree. *Next* is the next element of the designator list.

(FIELDREF Ident *Next* Pos)

FIELDREF nodes represent a record field reference. *Next* is the next element of the designator list.

(BINARY Op Pos *Left Right*)

Left and *Right* are the expression trees for the left and right hand side of the operator. "Op" is the operator.

(UNARY Op *Right Right*)

Right is the expression tree for the right hand side of the operator. "Op" is the operator.

B Examples

```
PROGRAM P;
BEGIN
  x := y;
END.
```



```
<block>
<PROGRAM name="P" pos="1">
  <DECLNULL pos="2"/>
  <STATS pos="3">
    <ASSIGN pos="3">
      <VARREF ident="x" pos="3">
        <DESNNULL pos="3"/>
      </VARREF>
      <VARREF ident="y" pos="3">
        <DESNNULL pos="3"/>
      </VARREF>
    </ASSIGN>
    <STATNULL pos="4"/>
  </STATS>
</PROGRAM>
</block>
```

```
PROGRAM P;
BEGIN
  IF x THEN
    WRITELN;
  ENDIF;
END.
```



```
<block>
<PROGRAM name="P" pos="1">
  <DECLNULL pos="2"/>
  <STATS pos="5">
    <IF1 pos="3">
      <VARREF ident="x" pos="3">
        <DESNNULL pos="3"/>
      </VARREF>
      <STATS pos="4">
        <WRITELN pos="4"/>
        <STATNULL pos="5"/>
      </STATS>
    </IF1>
    <STATNULL pos="6"/>
  </STATS>
</PROGRAM>
</block>
```

```
PROGRAM P;
TYPE x = ARRAY a OF T;
BEGIN
END.
```



```
<block>
<PROGRAM name="P" pos="1">
  <DECLS pos="2">
    <ARRAYDECL ident="x" elementType="T" pos="2">
      <VARREF ident="a" pos="2">
        <DESNNULL pos="2"/>
      </VARREF>
    </ARRAYDECL>
  <DECLNULL pos="3"/>
</DECLS>
  <STATNULL pos="4"/>
</PROGRAM>
</block>
```

```

PROGRAM P;
BEGIN
    WRITE a*b/c;
END.

```



```

<block>
<PROGRAM name="P" pos="1">
  <DECLNULL pos="2"/>
  <STATS pos="3">
    <WRITE pos="3">
      <BINARY op="SLASH" pos="3">
        <BINARY op="STAR" pos="3">
          <VARREF ident="a" pos="3">
            <DESNNULL pos="3"/>
          </VARREF>
          <VARREF ident="b" pos="3">
            <DESNNULL pos="3"/>
          </VARREF>
        </BINARY>
        <VARREF ident="c" pos="3">
          <DESNNULL pos="3"/>
        </VARREF>
      </BINARY>
    </WRITE>
    <STATNULL pos="4"/>
  </STATS>
</PROGRAM>
</block>

```

```

PROGRAM P;
BEGIN
    x(y);
END.

```



```

<block>
<PROGRAM name="P" pos="1">
  <DECLNULL pos="2"/>
  <STATS pos="3">
    <PROCCALL pos="3">
      <VARREF ident="x" pos="3">
        <DESNNULL pos="3"/>
      </VARREF>
      <ACTUAL pos="3">
        <VARREF ident="y" pos="3">
          <DESNNULL pos="3"/>
        </VARREF>
        <ACTUALNULL pos="3"/>
      </ACTUAL>
    </PROCCALL>
    <STATNULL pos="4"/>
  </STATS>
</PROGRAM>
</block>

```